Firmware Implementation Overview

Arduino Nano relay reset firmware

Codex

June 7, 2026

Table of Contents

Purpose **3**

Big Picture **3**

Hardware Defaults **4**

Serial Protocol **5**
- PING . . . 6
- STATUS . . . 6
- RESET . . . 7
- Unknown Or Long Commands . . . 8

State Machine **8**
- Ready . . . 8
- Pulsing . . . 9
- Cooldown . . . 9

Why The Pulse Is Non-Blocking **9**

Arduino Integration **10**
- Clock Callback . . . 10
- Relay Callback . . . 10
- Serial Callback . . . 11
- Setup . . . 11
- Loop . . . 11

Test Design **12**
- Fake Test Harness . . . 12
- Covered Behavior . . . 12

Real Hardware Testing **13**
- Embedded Unity Tests . . . 13
- Serial Smoke Test . . . 13

Safety And Failure Behavior **14**

Host-Side Expectations **14**

Current Verification **15**

Summary **15**

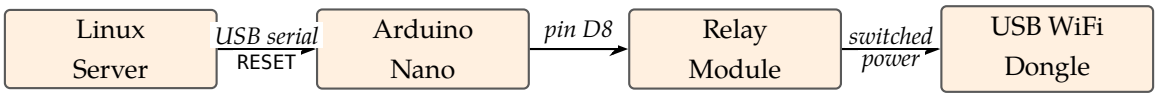
Purpose

This firmware turns a small Arduino Nano into a controlled relay pulser.

The practical goal is simple: a Linux server can send a serial command to the Arduino, and the Arduino briefly switches a relay. That relay can cut or switch power to a flaky USB WiFi dongle, causing the dongle to restart without rebooting the whole server.

The firmware is intentionally small, predictable, and easy to test. It does not try to be a general automation controller. It only accepts a few plain text commands and protects the relay from accidental repeated pulses.

The end-to-end signal chain looks like this:



A single `RESET` command travels from the server to the Arduino, which briefly switches the relay and power-cycles the dongle.

Big Picture

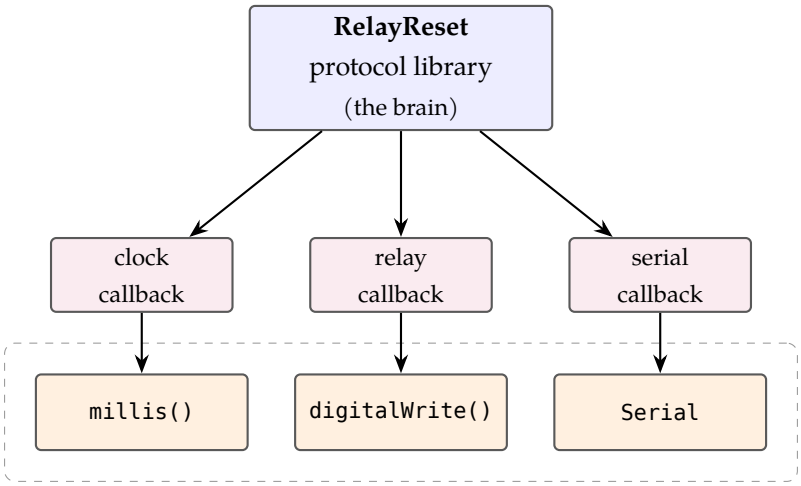
The project is split into two main parts:

- The Arduino-facing sketch in `src/main.cpp`
- The reusable protocol logic in `lib/RelayReset`

The sketch is the thin hardware layer. It knows about Arduino functions such as `Serial`, `millis()`, `pinMode()`, and `digitalWrite()`.

The protocol library is the actual brain. It knows how to read commands, track whether the relay is ready or busy, start a pulse, end a pulse, and answer status queries.

The two layers talk to each other through three callbacks, which is exactly what lets the protocol run either on real hardware or against fakes in a test:



This split is important because the protocol library can be tested on a normal computer without an Arduino attached. The tests provide a fake clock, fake relay, and fake serial output. That lets us test timing behavior instantly and deterministically.

Hardware Defaults

The current firmware targets:

- Board: Arduino Nano ATmega168
- PlatformIO board ID: `nanoatmega168`
- Framework: Arduino
- Serial baud rate: `9600`
- Relay pin: digital pin `D8`
- Relay polarity: active-high
- Relay contact: normally-closed (USB powered by default)
- Pulse duration: `1000 ms`
- Cooldown duration: `10000 ms`

These values are configured in two places.

The board and build settings live in `platformio.ini`:

```
[env:nanoatmega168]
platform = atmelavr
board = nanoatmega168
framework = arduino
test_framework = unity
monitor_speed = 9600
build_flags =
  -D RELAY_PIN=8
```

```
  -D RELAY_ACTIVE_LOW=0
```

The pulse and cooldown durations live in `src/main.cpp`:

```
static const RelayResetProtocol::Config kProtocolConfig = {
    1000,  // pulseMs
    10000, // cooldownMs
};
```

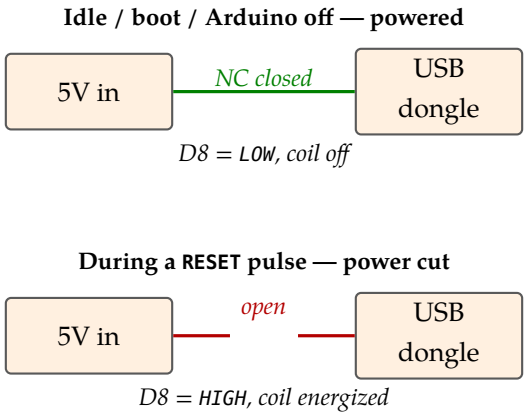
The relay is currently treated as active-high. That means the relay is activated (energized) when the pin is driven `HIGH`, and deactivated (de-energized) when the pin is driven `LOW`.

Relay wiring is fail-safe: it conducts by default

The relay module is wired through its **normally-closed (NC)** contact, so the USB WiFi dongle is powered whenever the relay is *not* energized — which is the default state while the Arduino is unpowered, booting, or simply idle.

Energizing the relay opens the dongle's power line. An active pulse drives `D8 HIGH`, the coil pulls in, the NC contact opens, and the dongle loses power for the pulse duration. When the pulse ends the pin returns `LOW`, the contact closes again, and power is restored. The practical consequence is that a failure of the Arduino (crash, reset, lost power, or never receiving a command) leaves the dongle *powered*, not cut off. Power is only ever interrupted by a deliberate, time-bounded pulse.

The two relay states look like this:



Serial Protocol

The firmware accepts line-based ASCII commands over serial.

The Linux host sends one command followed by a newline:

```
COMMAND\n
```

The firmware also accepts Windows-style line endings:

```
COMMAND\r\n
```

The carriage return character, `\r`, is ignored. The newline character, `\n`, marks the end of a command.

The supported commands are:

```
PING
STATUS
RESET
```

Unknown commands return an error.

PING

`PING` is a lightweight health check.

Request:

```
PING\n
```

Response:

```
PONG\n
```

This is useful for the host-side script because it can check that the Arduino is alive before asking it to do anything more serious.

STATUS

`STATUS` tells the host what the firmware is currently doing.

Request:

```
STATUS\n
```

Possible responses:

```
OK READY\n
OK PULSING <remaining-ms>\n
OK COOLDOWN <remaining-ms>\n
```

`OK READY` means the firmware is idle and will accept a reset command.

`OK PULSING` means the relay is currently active. The number tells how many milliseconds remain in the pulse.

`OK COOLDOWN` means the pulse has finished, but the firmware is intentionally refusing new reset commands for a short period. The number tells how many milliseconds remain before another reset is allowed.

RESET

`RESET` starts a relay pulse if the firmware is ready.

Request:

```
RESET\n
```

Normal response:

```
OK RESETTING\n
```

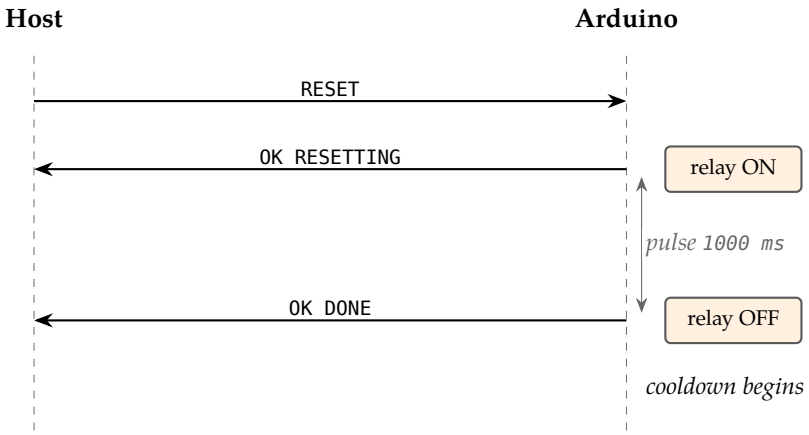
When the pulse ends, the firmware later writes:

```
OK DONE\n
```

That means a complete reset looks like this on the serial line:

```
host -> RESET\n
arduino -> OK RESETTING\n
arduino -> OK DONE\n
```

As a sequence over time:



If another reset command arrives while a pulse is already running, the firmware does not restart the timer and does not extend the pulse. It replies:

```
ERR BUSY <remaining-ms>\n
```

If a reset command arrives during cooldown, the firmware replies:

```
ERR COOLDOWN <remaining-ms>\n
```

This is meant to protect the relay and the WiFi dongle from a broken host script that accidentally sends many reset commands in a loop.

Unknown Or Long Commands

If a command is not recognized, the firmware replies:

```
ERR UNKNOWN\n
```

If a command line grows beyond the fixed command buffer size, the firmware replies:

```
ERR TOO_LONG\n
```

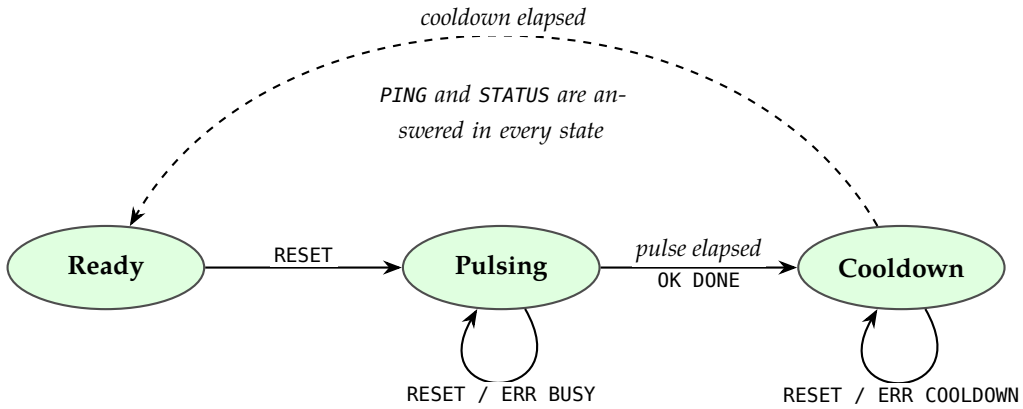
The command buffer is intentionally small:

```
static const size_t kMaxCommandLength = 24;
```

This is enough for the current protocol and avoids dynamic memory allocation on the small ATmega168.

State Machine

Internally, the firmware has three states:



In short: `Ready -> Pulsing -> Cooldown -> Ready`, with reset commands rejected (but not lost) while a pulse is running or cooling down.

Ready

In `Ready`, the relay is inactive and the firmware can accept a `RESET` command.

Valid behavior in this state:

- `PING` returns `PONG`
- `STATUS` returns `OK READY`

- `RESET` activates the relay and moves to `Pulsing`

Pulsing

In `Pulsing`, the relay is active. The firmware remembers when the pulse started. On every loop, it checks whether enough time has passed.

When the configured pulse duration has elapsed:

1. The relay is deactivated.
2. The state changes to `Cooldown`.
3. The firmware writes `OK DONE`.
4. The cooldown timer starts.

Valid behavior in this state:

- `PING` still works
- `STATUS` returns `OK PULSING <remaining-ms>`
- `RESET` returns `ERR BUSY <remaining-ms>`

The important detail is that repeated `RESET` commands do not extend the pulse. The first valid reset owns the pulse duration.

Cooldown

In `Cooldown`, the relay is inactive, but the firmware is not ready to reset again yet.

Valid behavior in this state:

- `PING` still works
- `STATUS` returns `OK COOLDOWN <remaining-ms>`
- `RESET` returns `ERR COOLDOWN <remaining-ms>`

When the cooldown duration has elapsed, the firmware silently returns to `Ready`.

Why The Pulse Is Non-Blocking

The firmware does not use `delay()` for the relay pulse.

Instead, it uses a non-blocking design:

1. `RESET` records the current time and turns the relay on.
2. The main loop keeps running.
3. `tick()` checks whether the pulse duration has elapsed.
4. When the time is up, `tick()` turns the relay off.

This design matters because the Arduino can still answer `PING` and `STATUS` during a pulse. It also makes tests much easier. The test code can move a fake clock forward by exactly `999 ms`, check that the relay is still on, move it forward by `1 ms`, and check that the relay turned off.

With `delay(1000)`, the firmware would be simpler but less observable. It would not process serial input while waiting, and timing tests would have to wait in real time.

Arduino Integration

The Arduino sketch is short because most of the behavior lives in the protocol library.

The sketch gives the protocol object three callbacks:

- A clock callback
- A relay output callback
- A serial output callback

Clock Callback

The protocol library does not call `millis()` directly. Instead, the sketch gives it this function:

```
static uint32_t protocolMillis(void *context) {
  (void)context;
  return millis();
}
```

On the real Arduino, this returns the Arduino's millisecond counter.

In tests, the same role is filled by a fake clock. That is why tests can control time without sleeping.

Relay Callback

The sketch also tells the protocol how to activate and deactivate the relay:

```
static void writeRelay(void *context, bool active) {
  (void)context;
#if RELAY_ACTIVE_LOW
  digitalWrite(RELAY_PIN, active ? LOW : HIGH);
#else
  digitalWrite(RELAY_PIN, active ? HIGH : LOW);
#endif
}
```

The protocol only says "relay active" or "relay inactive". The sketch translates that into the correct electrical signal for the actual relay module.

That keeps relay polarity out of the protocol logic.

Serial Callback

The protocol writes responses through this callback:

```
static void writeSerial(void *context, const char *text) {
  (void)context;
  Serial.print(text);
}
```

In tests, this callback writes into a memory buffer instead of the real serial port.

Setup

The Arduino setup function prepares the relay pin, puts the protocol into a safe initial state, and starts serial:

```
void setup() {
  pinMode(RELAY_PIN, OUTPUT);
  protocol.begin();
  Serial.begin(kSerialBaud);
}
```

The call to `protocol.begin()` makes sure the relay is inactive on boot. This is important because Nano boards often reset when a host opens the serial port. Opening serial should not accidentally pulse the relay.

Loop

The loop reads all available serial bytes and passes them into the protocol:

```
void loop() {
  while (Serial.available() > 0) {
    protocol.receive((char)Serial.read());
  }

  protocol.tick();
}
```

`receive()` handles command parsing. `tick()` handles time-based state changes, such as ending a pulse or finishing cooldown.

Test Design

The tests are written with Unity, PlatformIO's supported C/C++ unit test framework.

The same test file can run in two useful ways:

- On the host computer using the `native` environment
- On the Arduino Nano test environment

The host tests are the main day-to-day tests. They do not need hardware.

The Nano test target is useful because it confirms the same test code can compile for the real embedded platform. If a board is connected, PlatformIO can also run the tests on the device.

Fake Test Harness

The test harness replaces three real-world pieces:

- Time
- Relay hardware
- Serial output

It contains:

```
uint32_t now;
bool relayActive;
uint8_t relayTransitions;
char output[512];
RelayResetProtocol protocol;
```

`now` is the fake clock. Tests change it manually.

`relayActive` records whether the relay would currently be on.

`relayTransitions` counts how many times the relay changed state.

`output` stores responses that would normally be printed over serial.

This makes the protocol easy to test without the board.

Covered Behavior

The tests cover:

- Status querying
- `PING`
- Starting a reset
- Ending a reset
- Exact pulse duration behavior

- Overlapping reset commands
- Reset spam mixed with other commands
- Status responses across the full lifecycle

The most important timing test checks this:

1. Send `RESET`.
2. Advance fake time by `999 ms`.
3. Confirm the relay is still active.
4. Advance fake time by `1 ms`.
5. Confirm the relay is inactive and `OK DONE` was written.

That proves the pulse respects its configured duration.

The overlapping pulse test checks this:

1. Send `RESET`.
2. Advance fake time by `400 ms`.
3. Send `RESET` again.
4. Confirm the response is `ERR BUSY 600`.
5. Confirm the relay did not transition again.
6. Confirm the first pulse still ends at the original `1000 ms` mark.

That proves command spam does not stretch the relay pulse.

Real Hardware Testing

There are two levels of real-hardware testing.

Embedded Unity Tests

This command compiles the Unity tests for the Nano without uploading or needing a connected board:

```
~/.platformio/penv/bin/pio test -e nanoatmega168 --without-uploading --without-testing
```

With a connected Nano, this command can upload and run the tests:

```
~/.platformio/penv/bin/pio test -e nanoatmega168
```

These tests exercise the protocol code on the embedded target.

Serial Smoke Test

The project also includes `tools/serial_smoke.py`.

This script talks to the actual uploaded firmware over serial. By default, it is non-destructive. It only checks:

- `PING`
- `STATUS`

Example:

```
~/.platformio/penv/bin/python tools/serial_smoke.py --port /dev/ttyUSB0
```

To trigger a real relay pulse, the script requires the explicit `--pulse` flag:

```
~/.platformio/penv/bin/python tools/serial_smoke.py --port /dev/ttyUSB0 --pulse
```

That is intentional. A test command should not cut power to hardware unless the person running it clearly asks for that.

Safety And Failure Behavior

The firmware is designed to fail inactive.

On boot:

1. The relay pin is configured as an output.
2. The protocol is initialized.
3. The relay is explicitly set inactive.
4. Serial is started.

The firmware does not pulse on startup. It only pulses after receiving a complete `RESET\n` command.

The command parser is also intentionally conservative:

- Empty lines are ignored.
- Unknown commands do not affect the relay.
- Long commands are rejected.
- Repeated reset commands are rejected while busy.
- Reset commands are rejected during cooldown.

This does not make the system cryptographically secure. Anyone with access to the serial device can still send `RESET`. The assumption is that the Linux host owns the local serial port and normal filesystem permissions will control access.

Host-Side Expectations

The future Linux-side script should treat the Arduino as a small command server.

A reasonable host reset flow would be:

1. Open the serial port.
2. Wait briefly, because opening serial may reset the Nano.
3. Send `PING`.
4. Confirm `PONG`.
5. Send `STATUS`.
6. Confirm the device is ready, or wait for cooldown to finish.
7. Send `RESET`.
8. Confirm `OK RESETTING`.
9. Wait for `OK DONE`.
10. Close the serial port.

The host script should also use a lock file. The firmware has its own protection, but host-side locking prevents two Linux processes from fighting over the same serial device.

Current Verification

The following checks were run successfully:

```
~/.platformio/penv/bin/pio test -e native
```

Result:

```
7 test cases: 7 succeeded
```

The firmware build also succeeds:

```
~/.platformio/penv/bin/pio run -e nanoatmega168
```

The embedded test target compiles:

```
~/.platformio/penv/bin/pio test -e nanoatmega168 --without-uploading --without-testing
```

Summary

The implementation is intentionally small and direct.

The Arduino sketch handles hardware. The protocol library handles behavior. The tests fake the hardware so the important logic can be tested without a board.

The most important design decision is the non-blocking state machine. It keeps the device responsive, makes timing testable, and prevents reset spam from stretching or stacking relay pulses.

For the intended use case, this gives the Linux server a simple and observable way to power-cycle a bad WiFi dongle while keeping the firmware easy to audit.